

Generate a complete, single-file Android application using Kotlin and Jetpack Compose. The application will function as a UPI expense tracker that automatically parses transaction details from SMS messages.

App Name: UpiExpense Tracker

Core Functional Requirements:

1. Security First - Biometric Lock:

- Upon launching the app, immediately trigger a biometric authentication prompt using the BiometricPrompt API.
- The main content of the app should only become visible after successful authentication.
- If biometric authentication is not available or fails, show a message and prevent access to the app's content.

2. SMS Permissions and Parsing:

- On the main screen, after biometric authentication, check for the READ_SMS permission.
- If the permission is not granted, display a button that allows the user to request it. Do not proceed until the permission is granted.
- Implement a BroadcastReceiver to listen for new incoming SMS messages (android.provider.Telephony.SMS_RECEIVED).
- Create a robust SMS parsing function using regular expressions (Regex) to extract the following details from UPI transaction messages:
 - Transaction Amount (e.g., Rs. 50.00, INR 1,234.56)
 - Merchant/Recipient Name
 - Transaction Type (debited/credited)
- The parser must handle various common SMS formats from different banks and apps. Here are some examples to support:
 - "Your a/c XXXX is debited for Rs. 250.00 on 25-09-2025 and credited to VPA MERCHANT.XYZ@bank."
 - "Transaction of INR 75.50 has been made to SOME_STORE from your account."
 - "You have spent Rs. 120.00 on GPay at ANOTHER_VENDOR."
 - "Rs. 500.00 credited to your account no. XXXX via UPI."

3. Data Storage and Management (Local & Secure):

- Use the **Room Persistence Library** for the local database.
- The database must be **encrypted** using the **Jetpack Security library**.
- Define a Room Transaction Entity with the following fields:
 - id (Int, auto-generated primary key)
 - timestamp (Long, time of transaction)
 - amount (Double)
 - merchant (String)
 - isDebit (Boolean, true for debited, false for credited)
 - userNote (String, nullable, for user's personal notes)

- When a new transaction SMS is parsed, a new Transaction entity should be created and saved to the encrypted Room database.
- 4. **User Interface (Jetpack Compose & Material 3):**
 - The UI should be clean, modern, and follow Material 3 design guidelines.
 - **Main Screen:**
 - Display a summary card at the top showing the "Total Expenses This Week."
 - Show a list of all transactions from the last 7 days in a tabular/columnar format.
 - Each row in the list must display:
 - Date of the transaction.
 - Merchant name.
 - Transaction amount, colored red for debits and green for credits.
 - If the list is empty, display a message like "No transactions recorded in the last 7 days."
 - When a user clicks on a transaction row, a dialog box should appear, allowing them to view and edit the userNote for that specific transaction.

Technical Architecture (MVVM):

- **Model:** The Room Transaction entity, TransactionDao, and the encrypted TransactionDatabase.
- **View:** All Jetpack Compose functions (@Composable). The main screen composable should observe the ViewModel.
- **ViewModel:** A TransactionViewModel that:
 - Holds the list of transactions for the past week in a StateFlow or LiveData.
 - Provides functions to addTransaction and updateTransactionNote.
 - Interacts with a TransactionRepository.
- **Repository:** A TransactionRepository class that acts as the single source of truth, abstracting the data source (the Room DAO).

Single-File Structure:

- All Kotlin code—including MainActivity, all @Composable functions, the TransactionViewModel, TransactionRepository, Transaction entity, TransactionDao, TransactionDatabase, and the SmsReceiver—**must be contained within a single .kt file**. This is a strict requirement.

Required Dependencies (for build.gradle.kts):

```
// Core & Compose
implementation("androidx.core:core-ktx:1.12.0")
implementation("androidx.lifecycle:lifecycle-runtime-ktx:2.6.2")
implementation("androidx.activity:activity-compose:1.8.2")
implementation(platform("androidx.compose:compose-bom:2023.08.00"))
implementation("androidx.compose.ui:ui")
implementation("androidx.compose.ui:ui-graphics")
implementation("androidx.compose.ui:ui-tooling-preview")
```

```

implementation("androidx.compose.material3:material3")

// ViewModel & LiveData for Compose
implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.6.2")
implementation("androidx.compose.runtime:runtime-livedata")

// Room Database
implementation("androidx.room:room-runtime:2.6.1")
implementation("androidx.room:room-ktx:2.6.1")
ksp("androidx.room:room-compiler:2.6.1")

// Security
implementation("androidx.security:security-crypto:1.0.0")
implementation("androidx.biometric:biometric-ktx:1.2.0-alpha05")

// Accompanist for Permissions
implementation("com.google.accompanist:accompanist-permissions:0.32.0")

```

AndroidManifest.xml Requirements:

- Ensure the following permissions are declared:


```

<uses-permission android:name="android.permission.READ_SMS" />
<uses-permission android:name="android.permission.RECEIVE_SMS" />
<uses-permission android:name="android.permission.USE_BIOMETRIC" />

```
- Register the SmsReceiver in the <application> tag:


```

<receiver android:name=".SmsReceiver" android:exported="true">
  <intent-filter>
    <action android:name="android.provider.Telephony.SMS_RECEIVED" />
  </intent-filter>
</receiver>

```

Please generate the code following these instructions precisely.